

Fruit Slice Game Features and Implementation Logic

1. BP_Target_Fruit (Fruit)

1.1 Overview and State Machine Design

Purpose

Implements the fruit slice game's core object — cuttable flying fruits. The GameManager takes a fruit from the object pool and activates it; it flies along a parabola in front of the player, and when the player's sword slices it the procedural mesh splits in two, the halves fly apart and score; if missed it is recycled on timeout. Fruits use a Procedural Mesh Component (FruitPMC) for their mesh because static meshes cannot be cut at runtime while procedural meshes can.

Fruits are not dynamically spawned and destroyed; instances in the object pool are reused. Each fruit's lifecycle is managed by the Fruit State state machine, and the Fruit Type enum distinguishes 8 types (lemon, orange, cucumber, watermelon, coconut, bread, cheesecake, mango); on activation the model and inner material switch randomly.

State machine design

- InPool: the inactive idle state — mesh hidden, no collision, no physics simulation, waiting for the GameManager to activate it
- Flying: activated, flying along the parabola, sliceable, with the lifetime timer counting
- Sliced: already cut; the two halves simulate physics and bounce apart, recycled after 2 seconds

1.2 Activate Event

Purpose

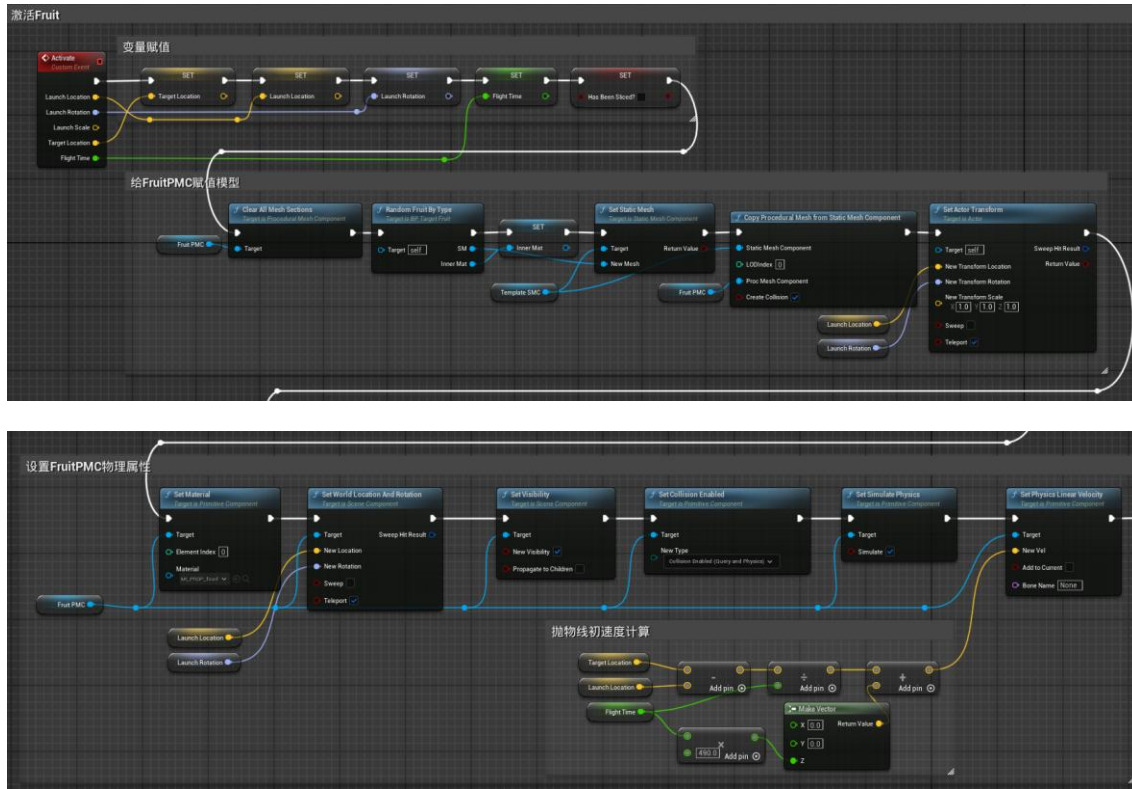
The fruit's launch entry. After the GameManager takes a fruit from the pool it calls this event, passing the launch position, launch rotation, target position, and flight time; inside, the event resets the mesh, randomly switches the model, initializes physics, and computes the parabolic initial velocity, finally launching the fruit. All input parameters are randomly generated by the GameManager's LaunchFruit logic.

Core principles

Parabolic initial velocity: computed separately for the horizontal and vertical directions and combined into one velocity vector.

- Horizontal velocity: $V_{xy} = (\text{TargetLocation} - \text{LaunchLocation}).XY / \text{FlightTime}$, guaranteeing the fruit reaches the target's exact X/Y coordinates after FlightTime seconds
- Vertical velocity: $V_z = 490 \times \text{FlightTime}$, where $490 = 980 / 2$ (half the gravitational acceleration), guaranteeing the fruit returns to exactly the launch height after FlightTime seconds
- Combined effect: the fruit flies along a symmetric parabola whose apex height $= 490 \times \text{FlightTime}^2 / 4$; standing in the middle of the arc's path, the player can slice it with the sword

Blueprint screenshots

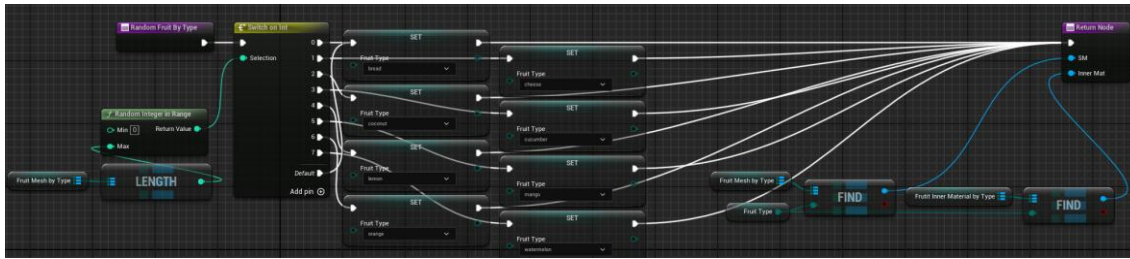


1.3 RandomFruitByType Function

Purpose

Randomly decides the fruit type on each activation and looks up the type's static mesh and inner material for Activate to use. Models and inner materials are stored in Maps: the key is the FruitType enum, and the values are a Static Mesh and a Material Instance.

Blueprint screenshots



Key points

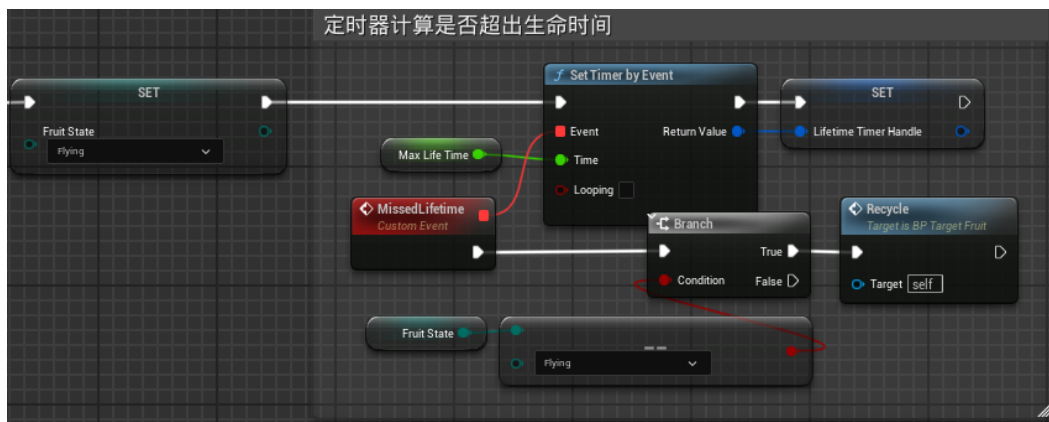
- The two Maps' key sets must match: adding a new fruit requires registering it in the enum, the Switch branches, and both Maps
- Inner materials are preconfigured in the Map and taken directly when slicing, avoiding runtime material creation

1.4 Lifetime Timer and MissedLifetime (Timeout Recycle) Event

Purpose

A fruit does not exist forever after activation: if the player misses it, it is automatically recycled after flying for MaxLifeTime seconds, preventing fruits from piling up in the scene or never being recycled after flying out of bounds.

Blueprint screenshots



Notes

- The state check cannot be skipped: if the fruit is already Sliced or back InPool, the timer expiry event still fires, so state filtering is required to avoid recycling an already-handled fruit twice

1.5 Collision Detection (On Component Begin Overlap)

Purpose

Slicing triggers when the fruit overlaps the sword's slice plane. This is the entry point for judging "the sword cut the fruit", filtered by three conditions so only valid slice events execute.

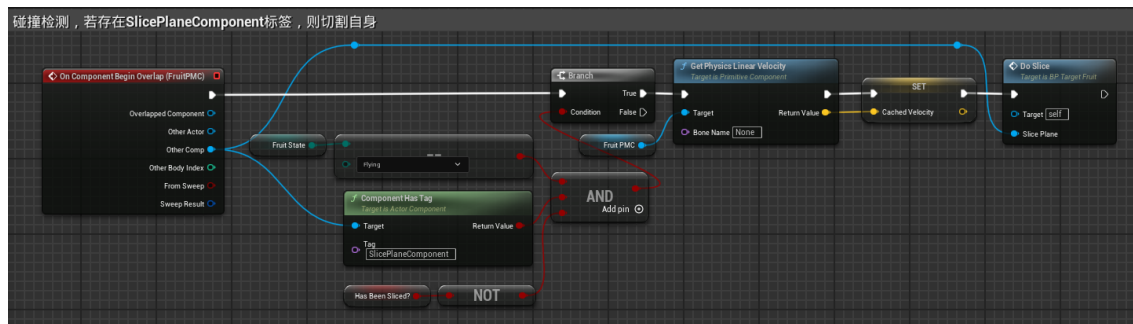
Trigger conditions (three conditions ANDed together; all must hold to slice)

- Condition 1: Fruit State == Flying — only flying fruits are sliceable (already-sliced or in-pool fruits do not respond)
- Condition 2: the other component Has Tag "SlicePlaneComponent" — only the sword's slice plane can trigger a cut; hitting the ground, walls, or other fruits does not
- Condition 3: NOT Has Been Sliced? — the slice plane may overlap the fruit several times in one frame; this flag prevents the same fruit from being sliced twice

Design notes

- The slice check does not verify swing speed: the sword's swing detection (Can Slice?) only controls the swing sound and does not participate in slicing — even a stationary sword touching a fruit can cut it. This deliberately forgiving design avoids the frustration of "seeing it but not cutting it" from overly strict speed thresholds in VR

Blueprint screenshots

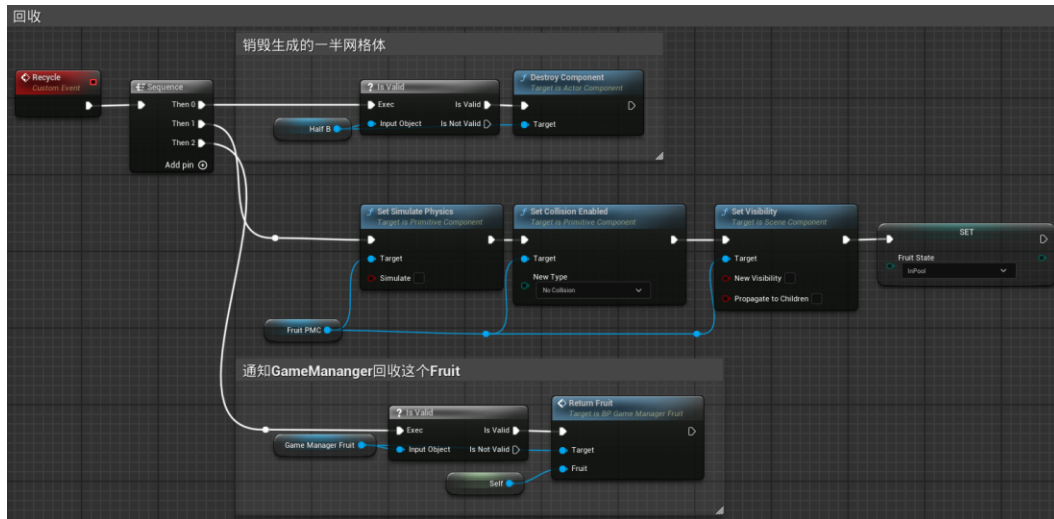


1.6 DoSlice Event

Purpose

The core of slicing: the slice plane's position and normal split the fruit's procedural mesh in two; the halves each inherit the original flight velocity plus a separation impulse bouncing them apart, while sound and effects play and score is added, with recycling after a 2-second delay.

Blueprint screenshots



2. BP_Weapon_Sword (Sword)

2.1 Overview and Inheritance Design

Purpose

The player's slicing weapon. It inherits from the weapon base class (same origin as the whack-a-mole hammer); all interaction abilities — VR grabbing, releasing, hand-following — come from the parent. The sword only extends two things on top: per-frame swing detection (speed and direction) and the swing sound.

Inheritance design

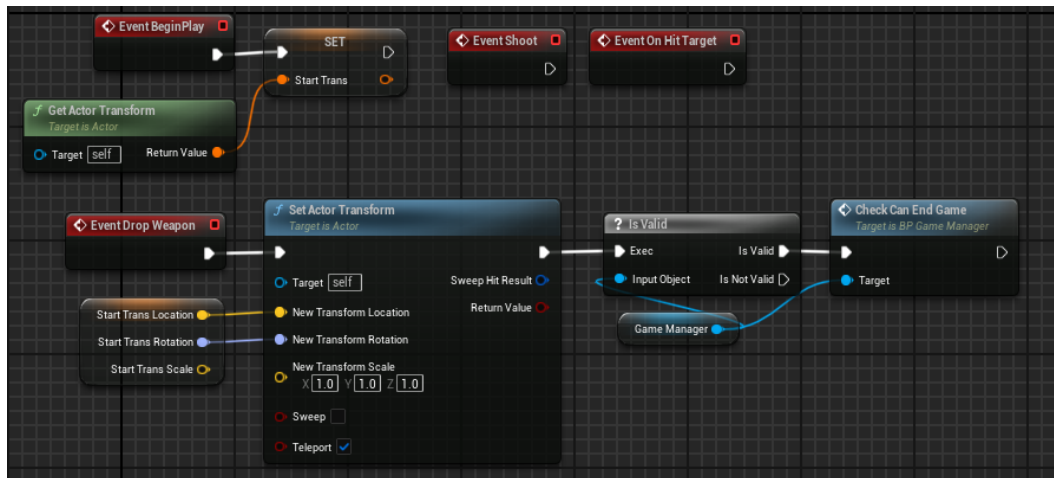
- Event Shoot and Event On Hit Target are overridden empty: these events serve the hammer's striking logic in the parent; the sword does not need them, so empty implementations override the parent events and block the default logic
- The sword's detection component is the Slice Plane, tagged SlicePlaneComponent; the fruit's Overlap identifies the sword through this tag

2.2 BeginPlay and DropWeapon (Return)

Purpose

Records the sword's initial placement; when the player drops it, it teleports back to the initial position and notifies the GameManager to check whether the game end condition is met (e.g. all weapons returned).

Blueprint screenshots

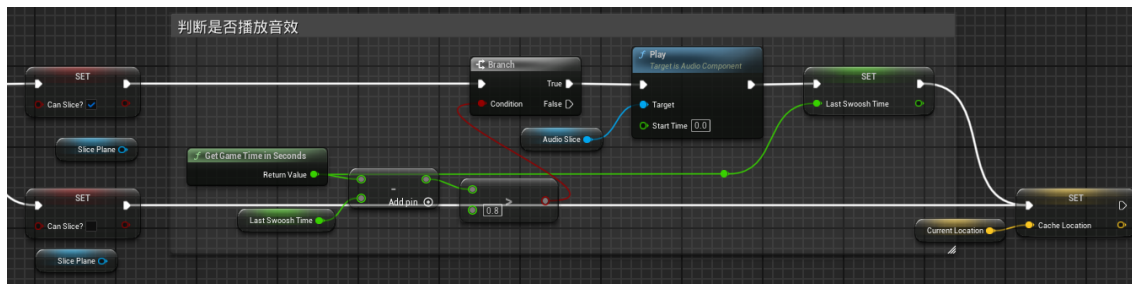
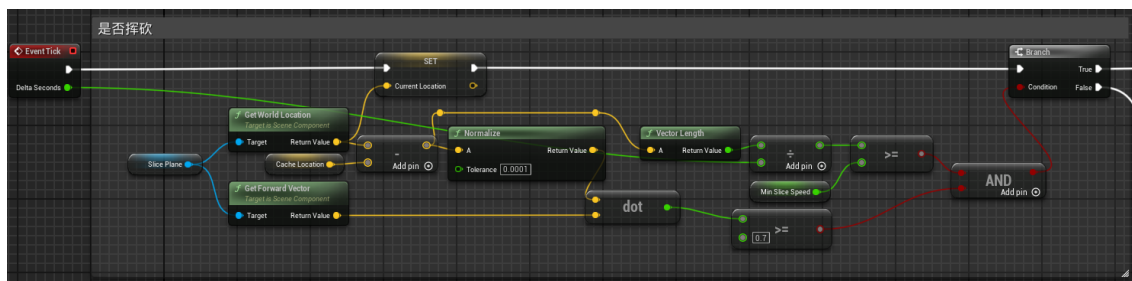


2.3 Tick Swing Detection (Speed and Direction)

Purpose

Every frame checks whether the sword is in a "valid swing" state: fast enough and with the swing direction matching the blade's facing. The result is written to the Can Slice? variable for the sound logic.

Blueprint screenshots



Key points

- Cache Location is updated after the detection logic (at frame end), ensuring the computation uses the previous frame's position — the standard pattern for inter-frame delta velocity

3. GameManager_Fruit (Game Manager)

3.1 Overview and Object Pool Design

Purpose

The fruit slice mode's manager (inheriting from the GameManager base class, isomorphic to the whack-a-mole manager): it creates and schedules the fruit object pool, launches fruits at a rhythm, clears the field on game end, and controls the game's end through the parent's countdown logic.

Core data structures

- Inactive Fruits (idle pool): the array of inactive fruits, taken from the head at launch
- Active Fruits: the array of flying/pending-recycle fruits, removed on recycle
- Launch Points: the Transforms of the left and right launch points, randomly chosen at launch
- Game Config: the configuration structure from the parent class, providing the initial delay, launch interval, and other parameters

Game parameters

Variable	Type	Description
InactiveFruits	BP_Target_Fruit array (public)	Idle pool: stores inactive fruits.
ActiveFruits	BP_Target_Fruit array (public)	Active array: stores activated fruits (flying / sliced and pending recycle)
FruitPoolSize	Integer	Object pool capacity

Variables (Spawn category)

Variable	Type	Description
SpawnInterval	Float	Launch interval: the looping timer's period; one fruit launches per interval
LaunchPoints	Scene Component array	Launch point array: holds the left and right launch points
TempLaunchPosition	Vector	Temporary launch position: the world position of the launch point randomly chosen this time
TempTargetPosition	Vector	Temporary target position: the landing point of the reference target position plus a random offset (± 15)
MinFlightTime	Float	Minimum flight time
MaxFlightTime	Float	Maximum flight time

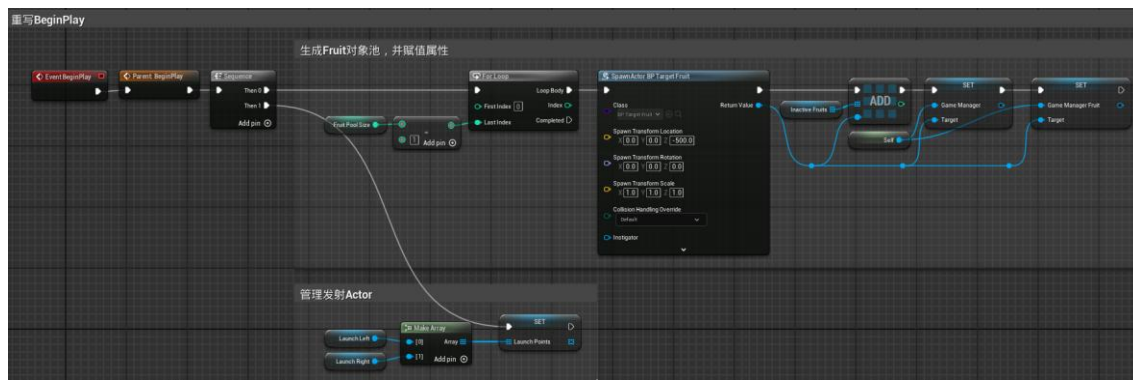
TempFlightTime	Float	Temporary flight time
TempFruit	BP_Target_Fruit object	Temporary fruit reference
IsGameRunning	Boolean	Game running flag

3.2 BeginPlay (Object Pool Initialization)

Purpose

Pre-generates the entire fruit object pool before the game starts and initializes the launch points. Pre-generating means launching a fruit is just "take and activate" with no runtime spawn cost, avoiding hitches.

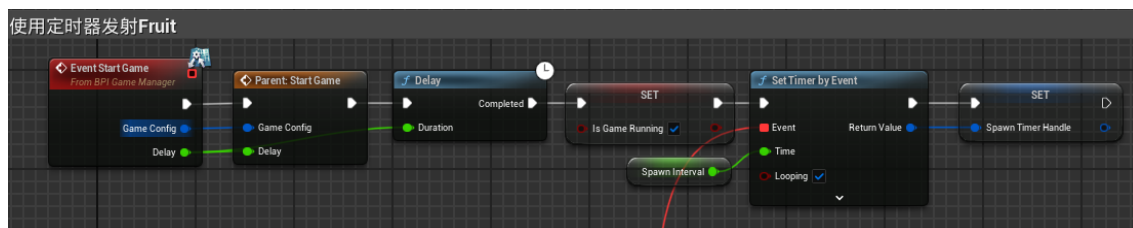
Blueprint screenshots



3.3 StartGame (Game Start and Launch Timer)

Purpose

Implements the BPI Game Manager interface's StartGame event: after waiting out the initial delay it starts the launch loop, with a looping timer continuously launching fruits at a fixed interval. Blueprint screenshots

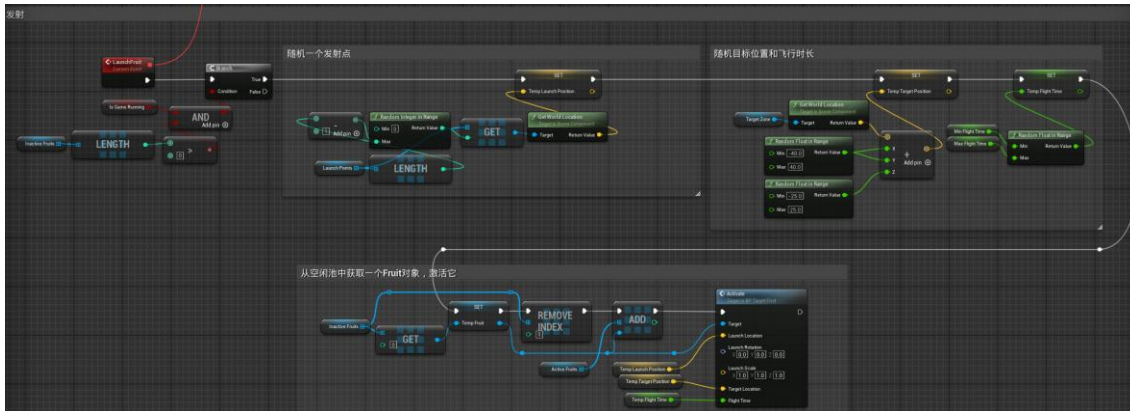


3.4 LaunchFruit (Launching a Fruit)

Purpose

Runs each time the launch timer fires: randomly decides the launch point, target position, and flight time, takes a fruit from the idle pool, and activates it.

Blueprint screenshots



3.5 ReturnFruit (Return to Pool)

Purpose

The final step of fruit recycling: removed from the active array and added to the idle pool, completing one full pool cycle. Called by the fruit's own Recycle event.

Blueprint screenshots



Key points

- Pool-entry safety is guaranteed on the fruit side: the Fruit State $\neq$ InPool check at the start of Recycle ensures each fruit reaches ReturnFruit only once and is never double-pooled

3.6 StopSpawnFruit (Stop Spawning and Clear the Field)

Purpose

On game end, stops launching and clears all fruits on the field: stop the timer, recycle every flying fruit, and empty the active array.

Blueprint screenshots

